The AI.DETECT_ANOMALIES function

Preview

This product or feature is subject to the "Pre-GA Offerings Terms" in the General Service Terms section of the Service Specific Terms (/terms/service-terms#1). Pre-GA products and features are available "as is" and might have limited support. For more information, see the launch stage descriptions (https://cloud.google.com/products/#product-launch-stages).

> **Note:** For support during the preview, contact bqml-feedback@google.com (mailto:bqml-feedback@google.com).

This document describes the **`AI.DETECT_ANOMALIES`** function, which lets you detect anomalies in time series data by using BigQuery ML's built-in TimesFM model (/bigquery/docs/timesfm-model).

For example, imagine you have historical and current data about sales of a product. You could run a query similar to the following to detect anomalous spikes or drops in sales:

```
SELECT *
FROM AI.DETECT_ANOMALIES(
  TABLE `mydataset.history_table`,
  TABLE `mydataset.target_table`,
  data_col => 'units_sold',
  timestamp_col => 'sales_date');
```

Syntax

```
SELECT
  *
FROM
  AI.DETECT_ANOMALIES(
    { TABLE HISTORY_TABLE | (HISTORY_QUERY_STATEMENT) },
    { TABLE TARGET_TABLE | (TARGET_QUERY_STATEMENT) },
    data_col => 'DATA_COL',
    timestamp_col => 'TIMESTAMP_COL'
    [, model => 'MODEL']
```

```
    [, id_cols => ID_COLS]
    [, anomaly_prob_threshold => ANOMALY_PROB_THRESHOLD]
  )
```

Description

The TimesFM model forecasts data for the `DATA_COL` value, based on the historical data provided in the `HISTORY_TABLE` or `HISTORY_QUERY_STATEMENT` argument, and using the fields contained in the `SELECT` statement as variables. The data provided by the `TARGET_TABLE` or `TARGET_QUERY_STATEMENT` argument is then compared to this forecasted data in order to detect anomalies. The tables or queries that provide the historical and target data must use the same column names for the `DATA_COL` and `TIMESTAMP_COL` arguments, and for the `ID_COLS` argument if it is used.

Arguments

`AI.DETECT_ANOMALIES` takes the following arguments:

- `HISTORY_TABLE`: the name of the table that contains historical time point data. For example, `` `mydataset.mytable` ``.

  If the table is in a different project, then you must prepend the project ID to the table name in the following format, including backticks:

  `` `[PROJECT_ID].[DATASET].[TABLE]` ``

  For example, `` `myproject.mydataset.mytable` ``.

- `HISTORY_QUERY_STATEMENT`: the query that generates the historical data.

- `TARGET_TABLE`: the name of the table that contains the data in which you want to detect anomalies. The table's schema must match the schema of the historical data.

  If the table is in a different project, then you must prepend the project ID to the table name in the following format, including backticks:

  `` `[PROJECT_ID].[DATASET].[TABLE]` ``

  For example, `` `myproject.mydataset.mytable` ``.

- `TARGET_QUERY_STATEMENT`: the query that generates the data in which you want to detect anomalies. The schema of the query result should match the schema of the historical data.

- `DATA_COL`: a `STRING` value that specifies the name of the data column. The data column contains the data to evaluate. The data column must use one of the following data types:
    - `INT64`
    - `NUMERIC`
    - `BIGNUMERIC`
    - `FLOAT64`
- `TIMESTAMP_COL`: a `STRING` value that specifies the name of the timestamp column. The timestamp column must use one of the following data types:
    - `TIMESTAMP`
    - `DATE`
    - `DATETIME`
- `MODEL_NAME`: a `STRING` value that specifies the name of the model to use. Supported models include `TimesFM 2.0` and `TimesFM 2.5`. The default value is `TimesFM 2.0`.
- `ID_COLS`: an `ARRAY<STRING>` value that specifies the names of one or more ID columns. Each ID identifies a unique time series to evaluate. Specify one or more values for this argument in order to evaluate multiple time series using a single query. The columns that you specify must use one of the following data types:
    - `STRING`
    - `INT64`
    - `ARRAY<STRING>`
    - `ARRAY<INT64>`
- `ANOMALY_PROB_THRESHOLD`: a `FLOAT64` value that specifies the custom threshold to use for anomaly detection. The value must be in the range `[0, 1)`. The default value is `0.95`.

  The value of the anomaly probability at each timestamp is calculated using the target time series data value, the historical time series data values, and the variance from the model training. The target time series data value at a specific timestamp is identified as anomalous if the anomaly probability exceeds the `ANOMALY_PROB_THRESHOLD` value. The `ANOMALY_PROB_THRESHOLD` value also determines the lower and upper bounds, where a larger threshold value results in a larger interval size.

Output

`AI.DETECT_ANOMALIES` returns the following columns:

- All time series ID columns, as specified in the `ID_COLS` argument.
- `time_series_timestamp`: a `STRING` value that contains the timestamp column for a time series. This value is inherited from the `TIMESTAMP_COL` argument.
- `time_series_data`: a `STRING` value that contains the data column for a time series. This value is inherited from the `DATA_COL` argument.
- `is_anomaly`: a `BOOL` value that indicates whether the value associated with a given time point is an anomaly.
- `lower_bound`: a `FLOAT64` value that contains the lower bound of the prediction result.
- `upper_bound`: a `FLOAT64` value that contains the upper bound of the prediction result.
- `anomaly_probability`: a `FLOAT64` value that contains the probability the value associated with a given time point is an anomaly.
- `ai_detect_anomalies_status`: a `STRING` value that contains the anomaly detection operation status. The value is empty if the operation was successful. If the operation wasn't successful, the value is the error string. A common error is `The time series data is too short.` This error indicates that there wasn't enough historical data in the time series to evaluate. A minimum of 3 data points is required.

If the input is invalid, such as if the time series length is too short, then the function returns `NULL` values in the `is_anomaly`, `upper_bound`, `lower_bound`, and `anomaly_probability` columns.

Examples

The following examples detect anomalies in the number of bike trips recorded in the New York Citibike trips table (https://console.cloud.google.com/bigquery?p=bigquery-public-data&d=new_york&t=citibike_trips&page=table) for two months following July 1, 2016.

This query shows how to detect anomalies in the number of trips that happened each day:

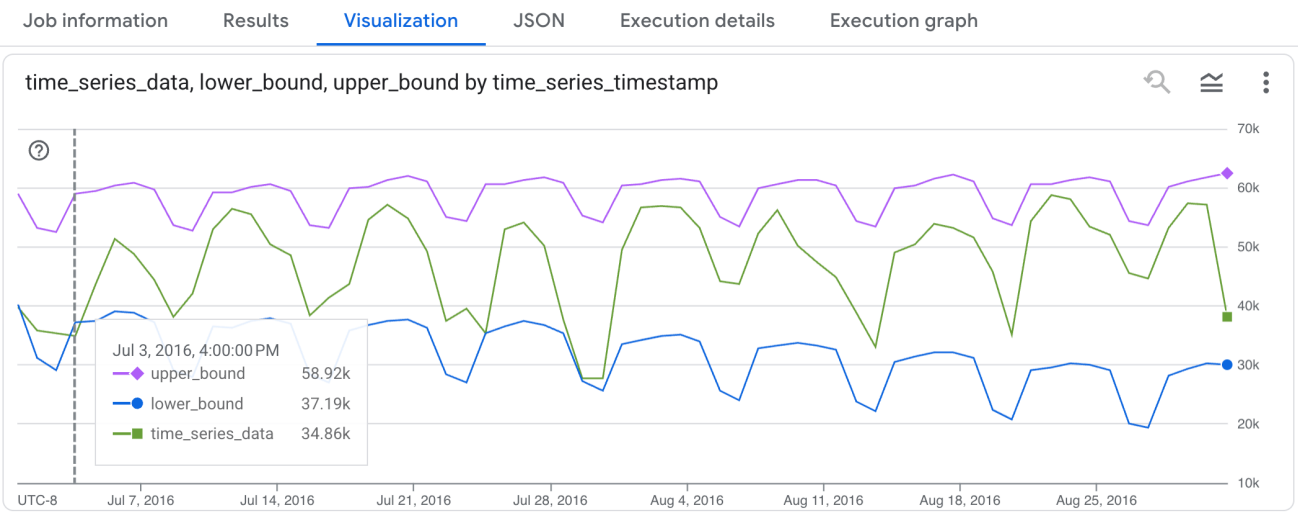
```
WITH bike_trips AS (
    SELECT EXTRACT(DATE FROM starttime) AS date, COUNT(*) AS num_trips
    FROM `bigquery-public-data.new_york.citibike_trips`
    GROUP BY date
  )
SELECT *
FROM
  AI.DETECT_ANOMALIES(
    # Historical data from a query
    (SELECT * FROM bike_trips WHERE date <= DATE('2016-06-30')),
    # Target data from a query
    (SELECT * FROM bike_trips WHERE date BETWEEN '2016-07-01' AND '2016-09-01
    data_col => 'num_trips',
    timestamp_col => 'date');
```

The first few rows of output look similar to the following:

```
+-------------------------+------------------+------------+-------------------
| time_series_timestamp   | time_series_data | is_anomaly | lower_bound
+-------------------------+------------------+------------+-------------------
| 2016-07-02 00:00:00 UTC | 35837.0          | false      | 31180.30030954070
| 2016-07-03 00:00:00 UTC | 35348.0          | false      | 29203.3735854947
| 2016-07-04 00:00:00 UTC | 34860.0          | true       | 37186.31338822422
+-------------------------+------------------+------------+-------------------
```

Anomalies occur when the **`time_series_data`** is below the **`lower_bound`** or above the **`upper_bound`**. To visualize the results, in the **Query results** pane click **Visualization**.

> **Note:** The time displayed in the visualization is shown in the **`America/Los_Angeles`** time zone.



You can specify `usertype` in the `id_cols` argument to detect anomalies broken down by the type of user, which can be `Subscriber` or `Customer`:

```
WITH
  bike_trips AS (
    SELECT EXTRACT(DATE FROM starttime) AS date, usertype, COUNT(*) AS num_tr
    FROM `bigquery-public-data.new_york.citibike_trips`
    GROUP BY date, usertype
  )
SELECT *
FROM
  AI.DETECT_ANOMALIES(
    # Historical data from a query
    (SELECT * FROM bike_trips WHERE date <= DATE('2016-06-30')),
    # Target data from a query
    (SELECT * FROM bike_trips WHERE date BETWEEN '2016-07-01' AND '2016-09-01
    data_col => 'num_trips',
    timestamp_col => 'date',
    id_cols => ['usertype'])
```

This query breaks down the anomalies by the dimensions `usertype` and `gender`, specifies that BigQuery should use the `TimesFM 2.5` model, and sets `anomaly_prob_threshold` to 0.8:

```
WITH bike_trips AS (
    SELECT EXTRACT(DATE FROM starttime) AS date, usertype, gender, COUNT(*) AS
    FROM `bigquery-public-data.new_york.citibike_trips`
    GROUP BY date, usertype, gender
  )
```

```
SELECT *
FROM
  AI.DETECT_ANOMALIES(
    # Historical data from a query
    (SELECT * FROM bike_trips WHERE date <= DATE('2016-06-30')),
    # Target data from a query
    (SELECT * FROM bike_trips WHERE date BETWEEN '2016-07-01' AND '2016-09-01
    data_col => 'num_trips',
    timestamp_col => 'date',
    id_cols => ['usertype', 'gender'],
    model => "TimesFM 2.5",
    anomaly_prob_threshold => 0.8);
```

Limitations

Only the most recent 1,024 time points are evaluated for anomalies. If you need to evaluate more data points, reach out to bqml-feedback@google.com (mailto:bqml-feedback@google.com).

Locations

`AI.DETECT_ANOMALIES` and the TimesFM model are available in all supported BigQuery ML locations (/bigquery/docs/locations#bqml-loc).

Pricing

`AI.DETECT_ANOMALIES` usage is billed at the evaluation, inspection, and prediction rate documented in the **BigQuery ML on-demand pricing** section of the BigQuery ML pricing (/bigquery/pricing#bqml) page.

What's next

- Try Detect anomalies with a TimesFM univariate model (/bigquery/docs/timesfm-anomaly-detection-tutorial).

- For information about anomaly detection in BigQuery ML, see Anomaly detection overview (/bigquery/docs/anomaly-detection-overview).

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-02-19 UTC.